



Interfaces funcionais



Interfaces funcionais

- Interfaces funcionais são interfaces que possuem um único método abstrato (não implementado)
- Apareceram pela primeira vez em Java 8
- Eles são especialmente preparados para serem usados com expressões lambda
- Eles podem ter métodos padrão definidos (eles têm implementação)

```
@FunctionalInterface
public interface SayMyName {
    void say(String name);
}
```



Exemplo de interfaces funcionais

- Na própria linguagem Java, há uma série de classes funcionais
- Além disso, também podemos criar nossas próprias interfaces funcionais

- Interface funcional predicada

```
Predicate checkEmpty = (value) -> value.isEmpty();  
System.out.println(checkEmpty.test("ola"));  
System.out.println(checkEmpty.test(""));
```



Expressões lambda e interfaces funcionais

Se tomarmos a interface que criamos como exemplo, poderíamos usá-la usando uma classe anônima ou uma expressão lambda. Obviamente, estamos interessados no segundo

- Classe anônima

```
SayMyName sayMyName1 = new SayMyName() {  
    @Override  
    public void say(String name) {  
        System.out.println(name);  
    }  
};  
sayMyName1.say("Santi");
```

- Expressão lambda

```
SayMyName sayMyName2 = (name) -> System.out.println(name);  
sayMyName2.say("Santi");
```



Introdução a expressões lambda



Expressões Lambda

- Apareceram pela primeira vez em Java 8
- Código mais compacto e mais limpo
- Eles substituem o conceito de classe anônima
- Eles permitem expressar as instâncias de interface funcional de uma forma compacta

Sintaxis:

```
(parametros) -> (corpo-lambda)
```

Exemplos:

```
listaAnimales.sort((animal1, animal2) -> (animal1.getPeso() - animal2.getPeso()));  
Predicate<Animal> pesoMaiorQue10 = (animal) -> animal.getPeso() > 10;  
pesoMayorQue10.test(unAnimal); // true/false
```



Dúvidas?

Tokio.